

Contents

연구 방향 2: 메모리 관점 파이프라이닝 분석 보고서	1
1. 메모리 파이프라이닝 개념	2
1.1 핵심 아이디어	2
1.2 관련 연구 기반	2
2. Alpamayo 추론 파이프라인 구조 분석	2
2.1 추론 흐름 (소스코드 분석)	2
2.2 서브모듈별 메모리 프로파일	3
2.3 핵심 질문 답변	3
2.4 시각화 참조	3
3. 순차 오프로딩 실험 결과	4
3.1 실험 설정	4
3.2 시스템 RAM 제약	4
3.3 Phase별 실효 결과	4
3.4 순차 오프로딩 피크 VRAM	6
3.5 시각화 참조	6
4. 비동기 전송 실험 결과	6
4.1 PCIe 대역폭 측정	6
4.2 합성 모듈 실험 (1/10 스케일)	6
4.3 Alpamayo 전송 시간 추정	7
4.4 VLM 레이어별 오프로딩 시나리오	7
4.5 전체 추론 오버헤드 추정	7
4.6 시각화 참조	8
5. 적용 가능성 판단	8
5.1 종합 판단: 이론적으로 가능, 실현에는 상당한 엔지니어링 필요	8
5.2 핵심 장애물	8
5.3 실현 가능한 전략 비교	8
6. 예상 효과 및 한계	9
6.1 메모리 파이프라이닝의 예상 효과	9
6.2 PCIe 대역폭의 현실적 한계	9
6.3 D2H 비대칭 대역폭 문제	9
6.4 핵심 한계	9
7. 시각화 참조	10
부록: 실험 데이터 파일	10
부록: 방향 1과의 연결	10

연구 방향 2: 메모리 관점 파이프라이닝 분석 보고서

실험일: 2026-02-25 환경: NVIDIA GeForce RTX 3080 Ti (12 GB VRAM), 15 GB System RAM, 4 GB Swap 모델: nvidia/Alpamayo-R1-10B (11.08B params, BF16 = 22.16 GB) PyTorch 2.8.0+cu128, CUDA 12.8, Transformers 4.57.1

1. 메모리 파이프라이닝 개념

1.1 핵심 아이디어

Alpamayo-R1-10B는 3단계 순차 파이프라인으로 추론을 수행한다:

1. **Vision Encoder** -> 이미지를 임베딩으로 변환
2. **VLM (Language Model)** -> Chain-of-Causation 추론 + 토큰 생성
3. **Expert/Diffusion Decoder** -> 궤적 디코딩 (Flow Matching 10 steps)

이 단계가 **순차적**이므로, 한 번에 하나의 모듈만 GPU에 올리고 완료 후 CPU로 반환하는 “모듈 스와핑 파이프라인”이 이론적으로 가능하다. 이를 통해 피크 VRAM을 전체 모델 크기(22.16GB)가 아닌 **가장 큰 단일 모듈 크기**로 제한할 수 있다.

1.2 관련 연구 기반

- **Demand Layering (RTSS 2022)**: 레이어 단위 순차 로딩으로 96.5% 메모리 절감 (14.8% 지연 오버헤드)
 - **FlexGen (ICML 2023)**: GPU/CPU/Disk 3단계 메모리 계층 활용 오프로딩
 - **Superpipeline (2024)**: 모델을 파티션 단위로 나누어 동적 GPU/CPU 관리
 - **PIPO (2025)**: RTX 3060 (6GB)에서 디스크-CPU-GPU 파이프라인으로 3.1배 처리량 향상
 - **Diffusers enable_model_cpu_offload**: 모델 단위 CPU-GPU 순차 오프로딩
-

2. Alpamayo 추론 파이프라인 구조 분석

2.1 추론 흐름 (소스코드 분석)

`sample_trajectories_from_data_with_vlm_rollout()` 메서드의 실행 순서:

```
[ + ]
|
v
(1) fuse_traj_tokens() --
|
v
(2) vlm.generate() -- Autoregressive CoT
| - Vision Encoder: -> ( )
| - Language Model: 36 Transformer layers
| - : sequences + past_key_values (KV Cache)
|
v
(3) step_fn() x 10 (Flow Matching Euler integration)
| - action_in_proj: noisy action -> expert token embeddings
| - expert: 36-layer Transformer (VLM KV Cache )
| - action_out_proj: hidden states -> predicted action
|
v
(4) action_space.action_to_traj() -- action
|
```

v
[: pred_xyz, pred_rot]

2.2 서브모듈별 메모리 프로파일

서브모듈	파라미터	BF16 크기	GPU 단위
Vision Encoder (vlm.model.visual)	576.4M	1.15 GB	YES
VLM Embedding (vlm.model.language_model.embed_tokens)	637.7M	1.28 GB	YES
VLM Transformer Layer (x36)	192.9M/layer	0.386 GB/layer	YES (개별)
VLM LM Head (vlm.lm_head)	637.7M	1.28 GB	YES
VLM 전체	8.80B	17.60 GB	NO (120GB)
Expert Backbone (expert)	2,279.2M	4.56 GB	YES
Action Projections	1.35M	0.003 GB	YES
Diffusion (Flow Matching)	0 (파라미터 없음)	0 GB	N/A
전체 합계	11.08B	22.16 GB	NO

2.3 핵심 질문 답변

Q1: VLM 추론 중에 Expert도 GPU에 있어야 하는가?

A: 아니오. VLM의 generate() 호출 중에는 Expert 모듈이 전혀 사용되지 않는다. VLM 추론은 vlm.generate()로 독립적으로 실행되며, Expert는 이후 step_fn()에서만 호출된다. 따라서 VLM 추론 동안 Expert를 CPU에 둘 수 있다.

Q2: Vision Encoder 완료 후 결과를 저장하고 VLM 단계로 넘길 수 있는가?

A: 제한적으로 가능. Vision Encoder는 vlm.generate() 내부에서 자동으로 호출된다. Qwen3-VL의 generate()는 pixel_values를 받아 내부적으로 vision encoding을 수행하므로, Vision Encoder만 별도로 분리하려면 generate() 호출 방식을 수정해야 한다. 다만, vision encoding의 출력(시각 임베딩)은 수 MB 수준의 텐서이므로 CPU에 저장 가능하다.

Q3: 어떤 텐서가 단계 간 전달되는가?

전달 구간	텐서	추정 크기
Vision -> VLM	image embeddings (시각 임베딩)	~2-5 MB
VLM -> Expert	past_key_values (KV Cache)	~ 1-2 GB (seq_len에 비례)
VLM -> Expert	sequences (생성된 토큰)	~수 KB
Expert -> Output	sampled_action (꺾적)	~수 KB

핵심 의존성: Expert는 VLM의 past_key_values (KV Cache)를 직접 참조한다. 이 KV Cache는 Expert의 step_fn() 실행 동안 GPU에 있어야 하며, 크기가 1-2GB로 상당하다. 따라서 Expert 실행 시 Expert(4.56GB) + KV Cache(~1-2GB) = **~5.5-6.5GB**가 GPU에 있어야 한다.

2.4 시각화 참조

- 그림 1: figures/01_pipeline_diagram.png - 추론 파이프라인 구조도 및 순차 오프로딩 도식

3. 순차 오프로딩 실험 결과

3.1 실험 설정

- 모델을 CPU에 로드 (device_map="cpu", low_cpu_mem_usage=True)
- 각 서브모듈을 순차적으로 GPU로 이동, 측정 후 CPU로 반환
- 0.5초 간격 VRAM 모니터링

3.2 시스템 RAM 제약

항목	값
전체 RAM	15.55 GB
가용 RAM	13.37 GB
Swap 전체	4.00 GB
Swap 가용	1.15 GB
모델 크기 (BF16)	22.16 GB
RAM에 적재 가능	NO (8.79GB 부족)
Swap 포함 적재 가능	NO (여전히 부족)

중요: 시스템 RAM이 15GB밖에 되지 않으므로, 22GB 모델을 CPU에 전부 올리는 것이 불가능하다. 실험에서는 low_cpu_mem_usage=True를 사용하여 HuggingFace의 메모리 매핑 기법으로 로딩했다 (memory-mapped safetensors). 이 방식에서는 실제로 접근하는 파라미터만 RAM에 로드되므로, 전체 22GB를 동시에 RAM에 올리지 않아도 된다.

3.3 Phase별 실효 결과

Phase 1: Vision Encoder (1.15 GB)

메트릭	측정값
CPU->GPU 전송 시간	0.374s
GPU->CPU 전송 시간	0.855s
VRAM (로딩 후)	1.164 GB
VRAM (피크)	1.403 GB
총 phase 시간	1.642s
Forward 추론	입력 형식 이슈로 실행 실패 (전송 메트릭은 유효)

분석: Vision Encoder (1.15GB)는 12GB VRAM에 쉽게 탑재 가능. CPU->GPU 전송 약 0.37초, GPU->CPU 전송 약 0.86초. 비대칭적인 전송 시간은 PCIe 3.0 환경(WSL2)의 특성으로 보인다.

Phase 2: VLM Language Model (17.60 GB) VLM 전체는 17.60GB로 12GB VRAM을 크게 초과하므로, 레이어 단위 분석을 수행했다.

메트릭	측정값
전체 VLM 크기	17.596 GB

메트릭	측정값
Transformer 레이어 수	36
레이어당 크기	0.386 GB
레이어당 파라미터	192.9M
평균 CPU->GPU 전송 시간/레이어	0.344s
평균 GPU->CPU 전송 시간/레이어	0.274s
추정 대역폭	1.12 GB/s
36레이어 총 순차 전송 시간 (추정)	22.23s
LM Head 크기	1.275 GB
LM Head 전송 시간	1.184s
Embedding 크기	1.275 GB
레이어당 피크 VRAM	0.394 GB

레이어 샘플 측정:

레이어	크기	CPU->GPU	GPU->CPU	VRAM
Layer 0	0.386 GB	0.351s	0.274s	0.394 GB
Layer 18	0.386 GB	0.341s	0.274s	0.394 GB
Layer 35	0.386 GB	0.339s	0.274s	0.394 GB

분석: 레이어별 전송 시간은 매우 균일하며, 단일 레이어(0.39GB)는 12GB VRAM에 매우 여유롭게 들어간다. 그러나 36개 레이어를 순차적으로 전송하면 총 22초의 전송 오버헤드가 발생한다. 이는 VLM의 generate() 내부에서 autoregressive하게 여러 번 모든 레이어를 순회해야 하므로, 실제 오버헤드는 이보다 훨씬 크다.

Phase 3: Expert / Trajectory Decoder (4.56 GB)

메트릭	측정값
Expert 크기	4.558 GB
Action Projections 크기	0.003 GB
CPU->GPU 전송 시간	0.890s
GPU->CPU 전송 시간	2.185s
추론 시간 (더미 forward)	0.338s
VRAM (로딩 후)	4.570 GB
VRAM (피크)	4.571 GB
총 phase 시간	3.545s
Forward 추론	SUCCESS

분석: Expert (4.56GB)는 12GB VRAM에 단독으로 탑재 가능하다. CPU->GPU 전송 약 0.89초, GPU->CPU 약 2.19초. Expert는 10-step Flow Matching 중 반복적으로 호출되므로, 한번 GPU에 로드하고 10 step 모두 실행한 후 CPU로 반환하는 것이 효율적이다. 실제 추론에서는 KV Cache(~1-2GB)도 함께 GPU에 있어야 하므로, 피크 VRAM은 약 5.5-6.5GB로 추정된다.

3.4 순차 오프로딩 피크 VRAM

각 phase에서의 피크 VRAM:

Phase	피크 VRAM
Vision Encoder	1.403 GB
VLM (단일 레이어)	0.394 GB
Expert + KV Cache (추정)	~5.5-6.5 GB

이론적 순차 오프로딩 피크: ~6.5 GB (12GB 이내)

3.5 시각화 참조

- 그림 2: figures/02_vram_timeline.png - 실험 중 VRAM 시계열
- 그림 5: figures/05_layerwise_analysis.png - 모듈별 상세 분석

4. 비동기 전송 실험 결과

4.1 PCIe 대역폭 측정

실제 시스템의 CPU-GPU 간 전송 대역폭을 측정했다:

전송 크기	H2D (CPU->GPU)	D2H (GPU->CPU)
1 MB	1.21 GB/s	3.75 GB/s
10 MB	9.63 GB/s	6.60 GB/s
100 MB	8.38 GB/s	2.67 GB/s
500 MB	7.54 GB/s	2.63 GB/s
1 GB	7.85 GB/s	2.73 GB/s
2 GB	7.77 GB/s	2.36 GB/s
4 GB	7.91 GB/s	2.20 GB/s

안정 대역폭 (>= 500MB): - CPU->GPU (H2D): **7.77 GB/s** - GPU->CPU (D2H): **2.48 GB/s**

관찰: H2D 대역폭은 ~7.8 GB/s로 안정적이지만, D2H 대역폭은 ~2.5 GB/s로 **H2D의 1/3 수준**이다. 이는 WSL2 환경에서의 PCIe 비대칭 특성으로 보인다. 이론적 PCIe 3.0 x16 대역폭(15.75 GB/s)의 약 50% (H2D) 및 16% (D2H) 수준이다.

4.2 합성 모듈 실험 (1/10 스케일)

시스템 RAM 제약으로 실제 모델을 사용할 수 없어, Alpamayo 서브모듈의 크기 특성을 1/10 스케일로 모방한 합성 모듈로 실험했다.

방법	총 시간	피크 VRAM
동기식 순차 전송	1.059s	0.465 GB
비동기 프리페치	0.693s	0.512 GB
스피드업	1.53x	+47.3 MB

분석: CUDA 스트림 기반 비동기 프리페치가 동기식 대비 **1.53배 빠르다**. 대신 VRAM이 약 47MB 더 필요한데, 이는 현재 모듈과 다음 모듈이 일시적으로 동시에 GPU에 올라가기 때문이다.

4.3 Alpamayo 전송 시간 추정

측정된 PCIe 대역폭을 바탕으로 실제 Alpamayo 모듈의 전송 시간을 추정했다:

모듈	크기	H2D 시간	D2H 시간	전송 은닉 가능
Vision Encoder	1.15 GB	0.148s	0.464s	YES (연산 > 전송)
VLM Single Layer	0.386 GB	0.050s	0.156s	YES (연산 > 전송)
VLM Embedding+Norm	0.28 GB	0.036s	0.113s	NO
VLM LM Head	1.28 GB	0.165s	0.516s	NO
Expert Full	4.56 GB	0.587s	1.839s	YES (10-step 연산 >> 전송)
Action Projections	0.003 GB	<0.001s	<0.001s	YES

4.4 VLM 레이어별 오프로딩 시나리오

VLM의 36개 Transformer 레이어를 순차적으로 오프로딩할 때:

방법	총 전송 시간	스피드업
동기식 (load -> compute -> offload 순차)	9.19s	1.0x
비동기 파이프라인 (compute + prefetch 중첩)	2.01s	4.58x

비동기 파이프라인에서는 현재 레이어의 연산과 다음 레이어의 프리페치를 중첩하므로, 전송 오버헤드의 대부분이 은닉된다.

4.5 전체 추론 오버헤드 추정

시나리오	전송 오버헤드
Baseline (전체 GPU, 오프로딩 없음)	0s (추론 273.79s)
동기식 순차 오프로딩	~33.6s
비동기 파이프라인 오프로딩	~26.4s

주의: 이 추정은 VLM generate()의 한 번 forward pass 기준이다. Autoregressive 생성에서는 매 토큰마다 모든 레이어를 순회하므로, 256 토큰 생성 시 레이어 전환이 $36 \times 256 = 9,216$ 회 필요하다. 이는 순수 전송 오버헤드가 수십~수백 초에 달할 수 있음을 의미한다. 그러나 KV Cache를 활용하면 각 디코딩 스텝에서 단일 토큰만 처리하므로 레이어당 연산량이 적어, 파이프라인 효율이 저하될 수 있다.

4.6 시각화 참조

- 그림 3: figures/03_strategy_comparison.png - 전략 비교
- 그림 4: figures/04_pcie_bandwidth.png - PCIe 대역폭 + 전송 시간 추정

5. 적용 가능성 판단

5.1 종합 판단: 이론적으로 가능, 실현에는 상당한 엔지니어링 필요

모듈 순차 오프로딩의 핵심 결론:

판단 항목	결과
Vision Encoder 순차 오프로딩	가능 (1.15GB, 12GB 내 여유)
VLM 모듈 단위 오프로딩	불가능 (16.44GB > 12GB)
VLM 레이어 단위 오프로딩	가능 (0.39GB/layer), 단 큰 오버헤드
Expert 순차 오프로딩	가능 (4.56GB + KV Cache ~1-2GB = ~6.5GB)
전체 파이프라인 피크 VRAM	~6.5 GB (Expert + KV Cache가 병목)
12GB 내 실행	이론적 가능

5.2 핵심 장애물

1. **VLM generate() 내부 구조:** HuggingFace의 `generate()`는 모든 레이어가 동일 디바이스에 있다고 가정한다. 레이어별 오프로딩을 적용하려면 `generate()`를 커스텀 구현하거나, `accelerate`의 `dispatch_model`을 사용해야 한다.
2. **fuse_traj_tokens()의 디바이스 의존성:** 방향 1에서 발견한 바와 같이, `masked_scatter` 연산은 모든 텐서가 동일 디바이스에 있어야 한다. 오프로딩 시 이 처리를 명시적으로 GPU에서 수행해야 한다.
3. **KV Cache GPU 상주 요구:** Expert의 `step_fn()`은 VLM의 `past_key_values`를 직접 참조한다. VLM `generate()` 완료 후 KV Cache를 GPU에 유지한 상태에서 Expert를 GPU에 올려야 하므로, Expert(4.56GB) + KV Cache(~1-2GB)가 동시에 GPU에 있어야 한다.
4. **Autoregressive 디코딩의 비효율:** VLM의 autoregressive 생성에서 매 토큰마다 모든 36개 레이어를 순회하므로, 레이어별 오프로딩 오버헤드가 토큰 수에 비례하여 증가한다.
5. **시스템 RAM 제약:** 15GB RAM으로는 22GB 모델 전체를 CPU에 올릴 수 없다. Memory-mapped safetensors로 부분 로딩이 가능하나, 오프로딩된 레이어를 CPU RAM에 유지하는 데에도 제약이 있다.

5.3 실현 가능한 전략 비교

전략	피크 VRAM	추정 추론 시간	구현 난이도	실현성
(A) FP16 Baseline	21.52 GB	273.79s	N/A	불가능 (VRAM 초과)
(B) 모듈 단위 순차 오프로딩 (BF16)	~16.44 GB	-	낮음	불가능 (VLM > 12GB)
(C) 레이어별 오프로딩 (BF16) + 동기식	~6.5 GB	~500-800s+	매우 높음	이론적 가능
(D) 레이어별 오프로딩 (BF16) + 비동기	~6.5 GB	~400-600s+	매우 높음	이론적 가능

전략	피크 VRAM	추정 추론 시간	구현 난이도	실현성
(E) INT4 양자화 + 모듈 순차 오프로딩	~5.5-6 GB	~300-400s	중간	가장 현실적 최적
(F) INT4 양자화 + 전체 GPU 탑재	~5.5 GB	~300-350s	낮음	

6. 예상 효과 및 한계

6.1 메모리 파이프라이닝의 예상 효과

- **피크 VRAM 감소:** 22.16GB -> ~6.5GB (70% 감소)
- **12GB GPU 실행 가능:** Expert + KV Cache (~6.5GB)가 최대 부하로, 12GB 내 실행 가능
- **단, 추론 시간 대폭 증가:** BF16 레이어별 오프로딩 시 수분 이상의 추론 시간 예상

6.2 PCIe 대역폭의 현실적 한계

측정된 PCIe 대역폭: - H2D: 7.77 GB/s (이론 최대의 ~50%) - D2H: 2.48 GB/s (이론 최대의 ~16%)

이 대역폭에서의 전송 오버헤드: - VLM 단일 레이어 (0.39GB): H2D 0.05s + D2H 0.16s = **0.21s/layer**
 - 36 레이어 x 256 토큰: 36 x 0.21 x 256 = **1,935s** (동기식 최악 케이스) - 비동기 파이프라인으로
 ~4.5x 절감해도: **~430s** 오버헤드

결론: BF16에서의 레이어별 오프로딩은 전송 오버헤드가 너무 크다. INT4 양자화로 레이어 크기를 1/4로 줄이면 전송 시간도 비례하여 감소하므로, 양자화와의 조합이 필수적이다.

6.3 D2H 비대칭 대역폭 문제

GPU->CPU (D2H) 대역폭이 CPU->GPU (H2D)의 1/3 수준이라는 것은 심각한 병목이다. 이는 WSL2 환경 특유의 문제일 수 있으며, 네이티브 Linux에서는 더 나은 결과가 예상된다. 그러나 오프로딩 전략에서 D2H 전송이 빈번하므로, 이 비대칭성을 고려한 설계가 필요하다.

가능한 완화 방안: - GPU->CPU 전송을 최소화 (예: 레이어 파라미터를 CPU에 유지하고 GPU에는 복사본만 전송) - 비동기 D2H 전송으로 연산과 중첩 - INT4 양자화로 전송 데이터량 자체를 줄임

6.4 핵심 한계

1. **Autoregressive 디코딩과의 상충:** 레이어별 오프로딩은 단일 forward pass에서는 효율적이나, autoregressive 생성에서 매 토큰마다 전체 레이어를 순회하므로 오버헤드가 극대화된다.
2. **KV Cache 메모리 관리:** VLM generate() 동안 KV Cache가 점진적으로 증가하며, 이를 GPU에 유지해야 한다. 긴 CoT 추론(256+ 토큰)에서 KV Cache가 수 GB에 달할 수 있다.
3. **구현 복잡도:** HuggingFace의 generate() 내부 로직을 수정해야 하므로, 유지보수 부담이 크다.
4. **실시간 추론 불가:** 원래 Alpamayo의 추론 지연은 99ms이다. 오프로딩을 적용하면 수분 이상으로 증가하므로, 실시간 자율주행에는 사용할 수 없다. 오프라인 평가/실험용으로만 의미가 있다.

7. 시각화 참조

그림	파일명	설명
1	figures/01_pipeline_diagram.png	Alpamayo 추론 파이프라인 구조 및 순차 오프로딩 도식
2	figures/02_vram_timeline.png	순차 오프로딩 실험 중 VRAM 사용량 시계열
3	figures/03_strategy_comparison.png	전략별 피크 VRAM + 전송 오버헤드 + Sync/Async 비교
4	figures/04_pcie_bandwidth.png	PCIe 대역폭 측정 + 모듈별 전송 시간 추정
5	figures/05_layerwise_analysis.png	모듈별 전송/추론 시간, VRAM, VLM 레이어 분석, Sync/Async 비교

부록: 실험 데이터 파일

파일	설명
sequential_offload_results.json	Part 2 순차 오프로딩 실험 결과
sequential_offload_vram_timeline.csv	Part 2 VRAM 시계열 (0.5초 간격)
async_transfer_results.json	Part 3 비동기 전송 실험 결과
async_transfer_vram_timeline.csv	Part 3 VRAM 시계열
test_sequential_offload.py	Part 2 실험 스크립트
test_async_transfer.py	Part 3 실험 스크립트
create_figures.py	Part 4 시각화 스크립트

부록: 방향 1과의 연결

방향 1 (On-Demand Layering)에서 발견한 핵심 사항: - device_map="auto"는 커스텀 토큰 처리(fuse_traj_tokens)와 비호환 - VLM (16.44GB)이 단독으로도 12GB VRAM 초과 - Expert는 VLM의 KV Cache를 직접 참조

본 연구 방향 2에서 추가로 밝혀진 사항: - Vision Encoder와 Expert는 각각 독립적으로 GPU에 탑재/실행 가능 (실측 확인) - VLM 개별 레이어(0.39GB)의 전송 시간은 0.34s (H2D) + 0.27s (D2H) (실측) - PCIe 대역폭은 H2D 7.77 GB/s, D2H 2.48 GB/s (실측) - CUDA 스트림 비동기 프리페치로 1.53x 스피드업 (합성 모듈 실험) - VLM 레이어별 비동기 파이프라인으로 4.58x 스피드업 추정

결합 전략 제안: INT4 양자화 (방향별 연구 필요) + 모듈 순차 오프로딩 (본 방향) + 비동기 파이프라인 (본 방향)의 조합이 12GB VRAM에서 Alpamayo를 실행하는 가장 현실적인 경로이다.